



Aplicação Web Interativa

HTML5 + CSS3 + JavaScript ES6 + Manipulação do DOM

Curso: Desenvolvimento de Sistemas Disciplina: Programação Web

Data de Entrega: 25/09/2026

Equipe: Grupos de 2 a 4 alunos

Professor:

1. 🎯 O DESAFIO

Cada grupo deverá desenvolver um **sistema web interativo**, utilizando exclusivamente HTML, CSS e JavaScript. O projeto deverá solucionar um problema real relacionado a uma área temática escolhida pela equipe.

O objetivo principal é aplicar de forma prática os conhecimentos consolidados em aula, construindo uma aplicação que possua:

- Interface estruturada e responsiva;
- Design organizado e acessível;
- Interação avançada com o usuário;
- Manipulação dinâmica do DOM;
- Tratamento de eventos nativos;
- Formulários com validação rigorosa;
- Manipulação de estruturas de dados (JS);
- Filtros e mecanismos de pesquisa;
- Exibição e cálculo de dados dinâmicos;
- Aplicação de regras de negócio;
- Tratamento de erros e exceções.

Atenção: O projeto não deverá ser apenas um site institucional ou estático. A proposta exigida é o desenvolvimento de uma verdadeira **aplicação web interativa e funcional**.

2. 👥 FORMAÇÃO DOS GRUPOS

- Grupos compostos estritamente por **2 a 4 alunos**.
- **Participação coletiva:** Todos os integrantes deverão participar ativamente do desenvolvimento do código.
- **Domínio do projeto:** Todos deverão conhecer integralmente o funcionamento do sistema. O professor realizará perguntas individuais durante a avaliação.
- **Exclusividade:** Dois grupos não poderão apresentar sistemas com a mesma proposta exata ou código idêntico.

3. 💡 TEMAS SUGERIDOS

Cada grupo deverá escolher um dos 20 temas listados abaixo ou propor um tema próprio aprovado pelo professor:

01. 🏥 **Clínica Médica:** Cadastro de pacientes, médicos e consultas.

02. 📖 **Biblioteca:** Cadastro de livros, usuários e empréstimos.

03. 🍽️ **Restaurante:** Cardápio, clientes e controle de pedidos.

04. 🛒 **Mercado:** Produtos, categorias e controle de estoque.

05. 🐾 **Pet Shop:** Animais, clientes e agendamento de serviços.

06. 🏋️ **Academia:** Alunos, planos e ficha de atividades.

07. 🏨 **Hotel:** Quartos, hóspedes e reservas.

08. 🚗 **Estacionamento:** Veículos, vagas e controle de entrada/saída.

09. 🎬 **Cinema:** Filmes, sessões e reserva de assentos.

10. 🎮 **Game Center:** Jogadores, jogos e placar de pontuações.

11. 💰 **Finanças Pessoais:** Receitas, despesas e categorias.

12. ✈️ **Agência de Turismo:** Destinos, clientes e viagens.

13. 🏠 **Imobiliária:** Imóveis, clientes e negociações.

14. 👤 **Recursos Humanos:** Funcionários, setores e cargos.

15. 🚌 **Transporte:** Veículos, passageiros e rotas/viagens.

16. 🎵 **Plataforma Musical:** Artistas, músicas e playlists.

17. 🛒 **Loja Virtual:** Produtos, clientes e carrinho de compras.

18. 🎓 **Sistema Acadêmico:** Alunos, disciplinas e notas/frequência.

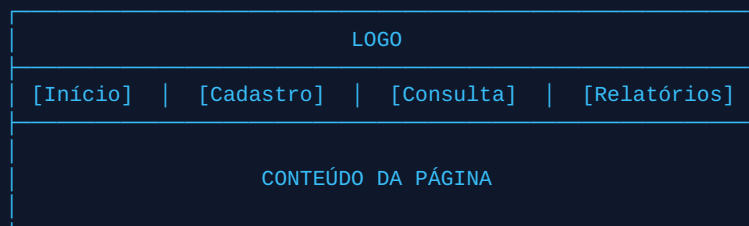
19. 🌱 **Reciclagem:** Materiais, pontos de coleta e usuários.

20. 📅 **Organização de Eventos:** Eventos, participantes e inscrições.

⚠️ **Tema próprio:** O grupo poderá propor outro tema não contido na lista, desde que submetido previamente à validação e aprovação do professor.

4. 🖥️ ESTRUTURA DO SISTEMA

O projeto deverá possuir no mínimo **4 telas ou seções funcionais** bem delineadas. A arquitetura de navegação sugerida é:



As telas/seções podem variar conforme o tema escolhido, por exemplo: *Dashboard, Cadastro, Consulta, Pesquisa, Relatórios, Perfil, Carrinho, Agendamento, Configurações*.

5. 🎨 REQUISITOS DE HTML E CSS

- **HTML Semântico:** Uso correto de tags como `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`.

- **Separação de responsabilidades:** Todo o código CSS deve estar em arquivo externo (ex: `style.css`).
- **Design & Usabilidade:** Layout limpo, hierarquia visual clara, botões e campos de formulário estilizados de forma padronizada.
- **Responsividade básica:** A interface deve se adaptar adequadamente para navegação em computadores e dispositivos móveis (celulares/tablets).

6, 7 & 8. REQUISITOS TÉCNICOS: JAVASCRIPT, DOM E EVENTOS

O JavaScript deve desempenhar papel central no funcionamento da aplicação. A avaliação verificará os seguintes mínimos quantitativos e qualitativos:

Categoria	Mínimo Exigido	Exemplos de Aplicação Prática
Funcionalidades JS	8 Mínimo	Cadastrar, excluir, editar, pesquisar, filtrar, calcular valores, alterar status, atualizar relatórios, gerar mensagens dinâmicas.
Manipulações DOM	8 Mínimo	Alterar <code>textContent</code> , <code>innerHTML</code> , modificar atributos e classes CSS, criar/remover elementos via <code>createElement/remove</code> , alternar visibilidade (<code>show/hide</code>), atualizar tabelas/cards.
Eventos Nativos	6 Mínimo	Escuta de eventos com propósito definido: <code>click</code> , <code>submit</code> , <code>input</code> , <code>change</code> , <code>mouseover</code> , <code>keydown</code> .

 **Regra de Ouro do DOM:** Não basta utilizar JavaScript apenas para emitir alertas com `alert("Olá!")`. O JavaScript deve modificar efetivamente e dinamicamente a estrutura e a interface visual da página.

9 & 10. FORMULÁRIOS E VALIDAÇÃO DE DADOS

A aplicação deve conter pelo menos **2 formulários funcionais** completos. Exemplo de estrutura visual mínima:

```
CADASTRO DE CLIENTE
Nome: [ ]
E-mail: [ ]
Telefone: [ ]
Categoria: [ Selecione uma opção ▼ ]
          [ Cadastrar ]
```

Validações obrigatórias (Mínimo de 8 regras de validação):

- Verificação de campos obrigatórios/vazios;
- Formato válido de e-mail;
- Bloqueio de números negativos ou fora da faixa permitida (mínimo/máximo);
- Prevenção de cadastros duplicados (ex: CPF, ID ou E-mail já existente);
- Tratamento de busca/consulta por registro inexistente;
- Seleção obrigatória em campos do tipo `<select>`.

Todas as mensagens de erro ou validação devem ser exibidas diretamente na interface (ex: banners, modais ou mensagens abaixo dos campos).

11 & 12. 📁 MANIPULAÇÃO DE DADOS, PESQUISA E FILTROS

Os dados do sistema devem ser armazenados e manipulados em memória através de **Arrays e Objetos** do JavaScript.

```
const alunos = [
  { nome: "Ana", idade: 17, curso: "DS" },
  { nome: "Kaue", idade: 18, curso: "DS" }
];
```

O sistema deve permitir realizar operações completas (CRUD) sobre essas estruturas de dados: *adicionar, consultar, editar, excluir, pesquisar, filtrar, contar e calcular*.

Ferramenta de Pesquisa e Filtro: Deverá permitir filtrar os resultados em tempo real utilizando pelo menos **2 critérios simultâneos** (ex: termo digitado + categoria selecionada + status).

13 & 14. 📊 DASHBOARD E RELATÓRIOS DINÂMICOS

A aplicação deve apresentar um **Dashboard** funcional com cards de métricas e uma seção dedicada a relatórios:



Regra Estrita: Os valores exibidos nos cards do Dashboard e nos relatórios devem ser calculados via JavaScript. É estritamente proibido fixar valores numéricos estáticos diretamente no HTML.

Relatórios exigidos (Mínimo de 3 indicadores dinâmicos):

- Quantidade total de registros;
- Contagem agregada por categoria ou status (disponíveis / indisponíveis);
- Cálculos financeiros ou estatísticos (maior valor, menor valor, média, total consolidado).

15 & 16. 🧠 REGRAS DE NEGÓCIO E EXCEÇÕES

O sistema deve implementar no mínimo **7 Regras de Negócio** inerentes ao contexto escolhido e tratar pelo menos **5 Situações de Exceção**.

Exemplo (Tema: E-commerce / Loja):

1. O produto deve obrigatoriamente possuir nome e preço válido.
2. O preço unitário não pode ser igual ou inferior a zero.
3. Produtos com estoque zerado não podem ser adicionados ao carrinho.
4. Apenas clientes cadastrados podem finalizar pedidos.

5. Produtos inexistentes não podem sofrer alteração de status.
6. Não é permitido finalizar a compra com o carrinho de compras vazio.
7. Descontos aplicados não podem ultrapassar 30% do valor total.

17. WIREFRAMES

Antes de iniciar a codificação, o grupo deverá desenhar os wireframes (rascunhos estruturais) das principais páginas. Deverão ser entregues no mínimo **4 wireframes**:

- Wireframe 01: Dashboard / Home
- Wireframe 02: Tela de Cadastro
- Wireframe 03: Tela de Consulta / Tabela
- Wireframe 04: Tela adicional escolhida pelo grupo

Ferramentas permitidas: Papel e caneta, Canva, Figma, PowerPoint, Google Slides, etc. O foco é o planejamento prévio do layout e usabilidade.

18. PLANO DE TESTES (15 TESTES)

A equipe deverá estruturar e documentar uma matriz com **15 testes funcionais** contendo a seguinte distribuição:

- 5 testes de funcionamento fluxo padrão (Happy Path);
- 5 testes de validação de entradas inválidas;
- 5 testes de regras de negócio e exceções.

Nº	Módulo	Entrada / Ação do Usuário	Resultado Esperado	Status
01	Cadastro	Preencher formulário com dados válidos	Cadastrar e exibir na tabela	OK
02	Cadastro	Deixar campo "Nome" em branco	Exibir mensagem de erro e bloquear	OK
03	Pesquisa	Buscar por termo existente	Filtrar e exibir registro correto	OK
04	Pesquisa	Buscar por termo inexistente	Exibir mensagem "Nenhum registro encontrado"	OK
05	Regra de Negócio	Tentar vender item sem estoque	Exibir aviso de estoque insuficiente	OK

19. ESTRUTURA DA DOCUMENTAÇÃO TÉCNICA

A documentação final do projeto deve seguir rigorosamente a estrutura numerada abaixo:

1. Capa Identificada
2. Introdução
3. Definição do Problema
4. Justificativa
5. Público-alvo
6. Objetivo Geral
7. Objetivos Específicos (Mín. 4)
8. Requisitos Funcionais (Mín. 8)
9. Requisitos Não-Funcionais (Mín. 4)
10. Regras de Negócio (Mín. 7)

11. Tecnologias Utilizadas
12. Modelagem dos Dados (JS)
13. Wireframes (4 telas)
14. Fluxograma do Sistema
15. Descrição das Funcionalidades

16. Matriz de Testes (15 testes)
17. Dificuldades Encontradas
18. Soluções Implementadas
19. Melhorias Futuras
20. Conclusão

20. 📁 ORGANIZAÇÃO DOS ARQUIVOS DO PROJETO

O repositório do projeto deve seguir uma estrutura de pastas limpa e padronizada:

```
PROJETO_WEB/
├── index.html           # Página principal / Dashboard
├── pages/
│   ├── cadastro.html   # Tela de formulários
│   ├── consulta.html   # Tela de pesquisas e tabelas
│   └── relatorios.html  # Tela de relatórios e estatísticas
├── css/
│   └── style.css        # Estilização centralizada
├── js/
│   └── script.js        # Scripts e lógica da aplicação
└── README.md           # Documentação resumida do repositório
```

21. 🤖 DIRETRIZES SOBRE USO DE INTELIGÊNCIA ARTIFICIAL

O uso de ferramentas de Inteligência Artificial é **permitido como recurso pedagógico de apoio** para:

- Esclarecer dúvidas conceituais e estudar sintaxe;
- Identificar e depurar erros de código (debugging);
- Pesquisar abordagens e boas práticas de estruturação;
- Auxiliar na redação da documentação e plano de testes.

🚫 **PROIBIÇÃO BÁSICA:** É estritamente proibido gerar o projeto inteiro via IA sem compreender o código e simplesmente entregá-lo. Todos os integrantes devem ser capazes de explicar cada linha de código.

Durante a arguição individual, o professor fará perguntas diretas como: "O que esta função executa?", "Por que este evento foi utilizado?", "Onde o DOM está sendo modificado?" ou "Faça uma alteração nesta funcionalidade agora."

"UTILIZAR IA PARA APRENDER É PERMITIDO. UTILIZAR IA SEM APRENDER NÃO É O OBJETIVO DO PROJETO."

22 & 23. 🎤 APRESENTAÇÃO E REGRAS GERAIS DE AVALIAÇÃO

Apresentação Prática (10 a 15 minutos por grupo):

- Exposição oral: Tema, Problema, Solução, Wireframes e Arquitetura.
- **Demonstração ao Vivo:** Execução das funcionalidades, validações, busca, cadastro e exclusão em tempo real perante a turma.
- **Teste de Estresse:** O professor solicitará ações específicas no momento da apresentação (ex: cadastrar dado em branco, buscar termo inexistente, simular erro de regra de negócio).

Regras Finais:

- O projeto deve funcionar 100% no ambiente local sem dependência de servidores Python ou bibliotecas externas complexas.
- Plágio ou cópias integrais de projetos entre grupos resultarão na invalidação da nota.
- Prazo Final de Entrega: **25/09/2026**.



DESAFIO FINAL

O objetivo deste projeto não é criar simplesmente uma página web, mas aprender a transformar uma ideia em uma aplicação interativa e profissional.

PLANEJAR → PROJETAR → PROGRAMAR → TESTAR → DOCUMENTAR → APRESENTAR